

Orbital AI -- RAG System Architecture (v2.1)

Section 1 -- Overview

Orbital AI's retrieval-augmented generation (RAG) system, codenamed 'Prometheus', is the internal knowledge layer that powers our support chatbot, the engineering wiki assistant, and the leadership briefing generator. It is built on a microservices architecture deployed across two regions (Bengaluru and Frankfurt) for redundancy.

The system is built on Python 3.11, LangChain 0.3, PostgreSQL 16 with the pgvector extension, and Google Gemini 2.0 Flash as the primary LLM.

Section 2 -- Indexing Pipeline

Documents enter the system through three ingestion paths:

- 1) Scheduled crawls of internal Confluence spaces (every 6 hours).
- 2) Webhook-triggered sync of the GitHub monorepo (on push to main).
- 3) Manual upload portal used by the operations team.

All documents are normalised to Markdown, chunked with a recursive character splitter (`chunk_size=800`, `overlap=150`), embedded with the BGE-small-en-v1.5 model, and stored in pgvector with HNSW indexing.

Section 3 -- Retrieval

At query time the system performs:

- * Query rewriting (HyDE + step-back)
- * Hybrid retrieval (BM25 + dense, fused with reciprocal rank)
- * Cross-encoder re-ranking (BGE-reranker-base)
- * Contextual compression to fit the prompt window

Average end-to-end latency: 480 ms p50, 1.2 s p95.

Section 4 -- Safety & Evaluation

Every response is evaluated by the offline RAGAS pipeline on a weekly-rotating golden set of 250 questions. Faithfulness, answer relevancy, and context precision/recall are tracked in Grafana. PII redaction happens at ingestion and again at generation.

